



FIAP PÓS-TECH • 7MLET • DATATHON • GRUPO 74

Adaptive Offers Platform

Decisão de ofertas financeiras em tempo real com multi-armed bandits contextuais e assistente LLM/RAG.

Bandit Contextual

Margin-Weighted

MLOps Ready

LGPD

Toda impressão é uma decisão de valor

● Regras fixas envelhecem

“Se idade > 60, mostre depósito.” Não aprendem com o comportamento e ficam obsoletas.

● Testes A/B são lentos e caros

Queimam semanas de tráfego na variante ruim até atingir significância estatística.

● Conversão $\neq$ lucro

Otimizar cliques ignora a margem e o contexto — você converte mais e ganha menos.

A PERGUNTA CENTRAL

Qual oferta mostrar para *este* cliente, *agora*, para maximizar o valor?

Um sistema que decide, aprende e se explica

1

Bandit contextual

Explora vs. explora e aprende online a cada decisão — sem esperar o fim de um A/B.

LinUCB · Thompson · UCB-V

2

Recompensa por margem

$\text{value} = P(\text{conversão}) \times \text{margem}.$
Otimiza lucro, não cliques.

Margin-Weighted Ranking

3

Assistente LLM + RAG

Resume experimentos e explica cada decisão com reason codes auditáveis.

Explicabilidade · offline

Base factual + camada sintética com oráculo conhecido

01

Base factual real

Bank Marketing real (UCI · 41.188 contatos públicos). Sem vazamento: duration removido.



02

Camada sintética

offer_catalog, offer_events e delayed rewards gerados por seeds reprodutíveis.



03

Oráculo → Regret

Como conhecemos o ótimo do gerador, medimos regret (distância do ótimo).

Por que sintético? recompensas atrasadas, oráculo mensurável e conformidade — nenhum dado pessoal real é usado (LGPD by design).

Quatro políticas competindo (+1 neural)

Baseline

Controle guloso — referência para medir o lift.

Thompson Sampling

Beta-Bernoulli. Exploração probabilística por amostragem.

Nilos-UCB (UCB-V)

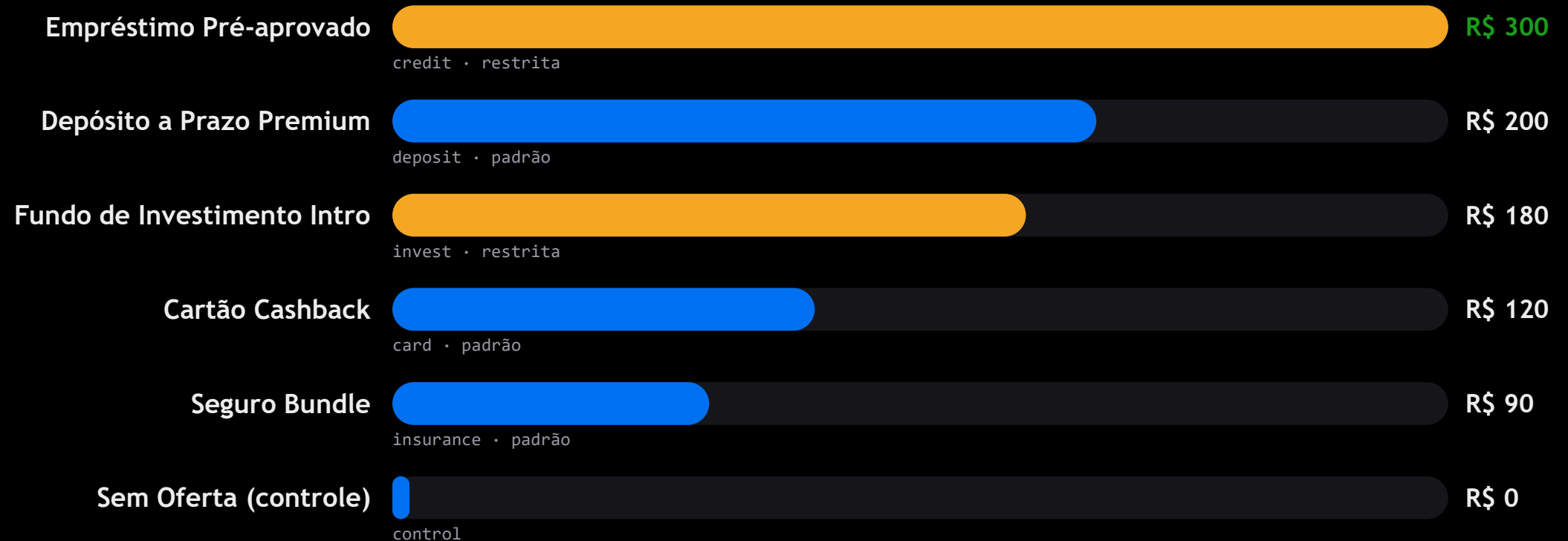
Otimismo consciente da variância do braço.

LinUCB (contextual)

Ridge linear sobre o contexto — menor regret nos dados reais.

+ Neural Bandit (PyTorch MLP, MC-dropout) como 5ª política opcional · cold-start e recompensas atrasadas tratados explicitamente.

6 ofertas, margens diferentes — por isso a margem importa



● restrita (gate de elegibilidade/suitability)

● padrão

Contexto → elegibilidade → política → log auditável



EXEMPLO REAL – MARGEM VENCE CONVERSÃO

Cliente 60 anos, com empréstimo ativo → escolhido: Fundo de Investimento (R\$ 15,4).

O Seguro tinha MAIOR conversão (10,9%), mas a margem baixa (R\$90) perdeu para o Fundo (8,6% × R\$180). A oferta de Empréstimo foi filtrada pela elegibilidade.

Oferta → canal → mensagem → próximo passo

O bandit decide *o quê*; quatro camadas determinísticas decidem *onde, como e o próximo passo*.

1 Segmentação comportamental

6 personas determinísticas das features reais (renegociador, sênior, jovem digital...). Leitura humana do book de clientes.

2 Orquestração multi-canal

App push · e-mail · SMS · voz com frequency cap e horário de silêncio.
Guardrail auditável com reason codes.

3 Next-Best-Action (NBA)

Mensagem por template governado + próximo passo (SIMULATE_LOAN, OPEN_DEPOSIT). Nunca texto livre do LLM.

4 IA responsável

Atributos protegidos + fairness por grupo: exposição 0,00 · valor 0,24 (≤30) — suitability, auditado, nunca decide.

Na base real, o aprendizado supera a regra fixa

+9,2%

melhor valor vs. baseline

8,3%

menor regret (LinUCB)

9,1%

maior conversão (LinUCB)

41.188

contatos reais (UCI)

VALOR ACUMULADO POR POLÍTICA (R\$ • 6.000 rodadas)

Thompson



R\$ 114.290

LinUCB



R\$ 113.230

Baseline



R\$ 104.700

Nilos-UCB



R\$ 102.020

A LEITURA HONESTA

Ganho real, mas modesto (~+8–9%) — realista, sem número mágico.

Baseline aposta 85% no Empréstimo; o LinUCB diversifica pelo contexto → **menor regret e maior conversão.**

Nem todo bandit vence: Nilos-UCB ficou abaixo do baseline (–2,6%).

Quatro superfícies demonstráveis — ao vivo

Decision Console

Next.js · :3000

- Explorador de decisão interativo
- Valor = $P \times \text{margem por oferta}$
- Explicação do assistente (LLM/RAG)

Swagger / OpenAPI

FastAPI · /docs

- Contrato interativo da API
- Testa POST /decide no navegador
- Schemas, exemplos e reason codes

Observability Board

Streamlit · :8503

- KPIs em tempo real + gauges
- Comparação entre políticas
- Feed de decisões auditáveis

MLflow Tracking

mlflow ui · :5001

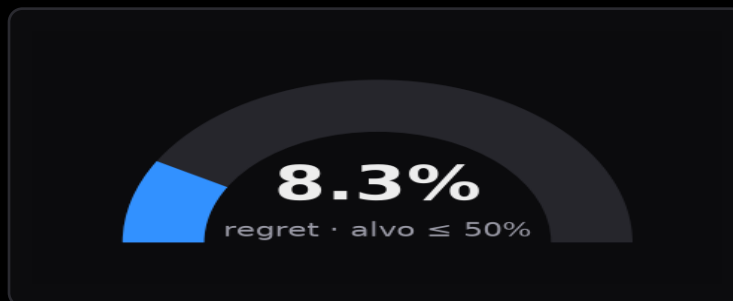
- Runs com params + métricas
- Comparação de experimentos
- Versões de política (registry)

O que cada gráfico te conta



Reward acumulado

tendência do valor capturado — subir é bom



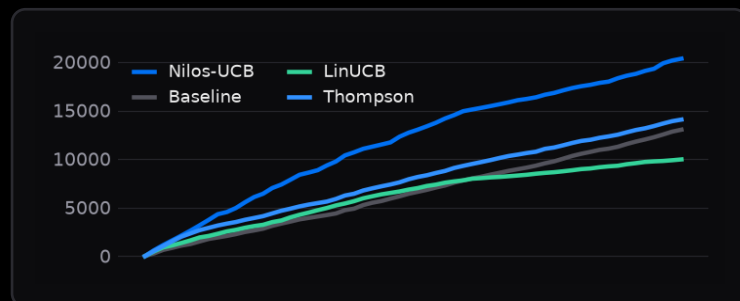
Regret ratio

distância do ótimo vs. alvo — menor é melhor



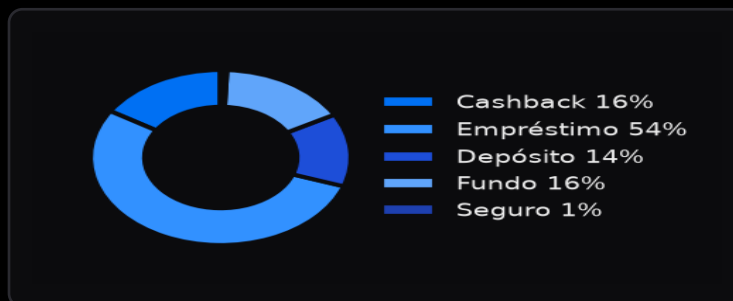
Valor por política

ranking de quanto cada algoritmo rende



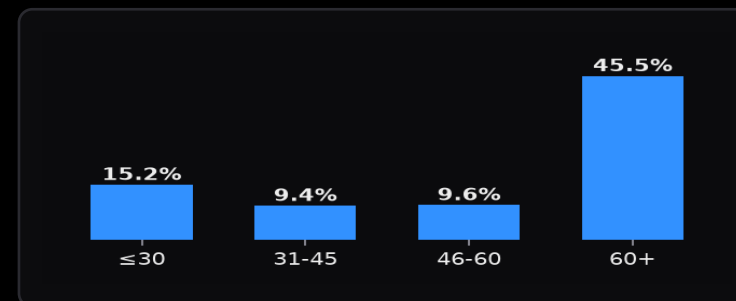
Regret acumulado

curva que achata = a política aprendeu



Mix de ofertas

distribuição das escolhas — detecta degeneração



Conversão por idade

sinal da base: qual perfil converte mais

Camadas claras, segurança gerenciada, escala a zero

COMPUTE

Azure Container Apps

API + jobs, escala a zero

DADOS

ADLS · Redis · PostgreSQL

feature store off/online

IA / LLM

Azure OpenAI · AI Search

RAG do assistente

MLOPS

Azure ML · MLflow

tracking + registry

OBSERVABILIDADE

App Insights

métricas, logs, traços

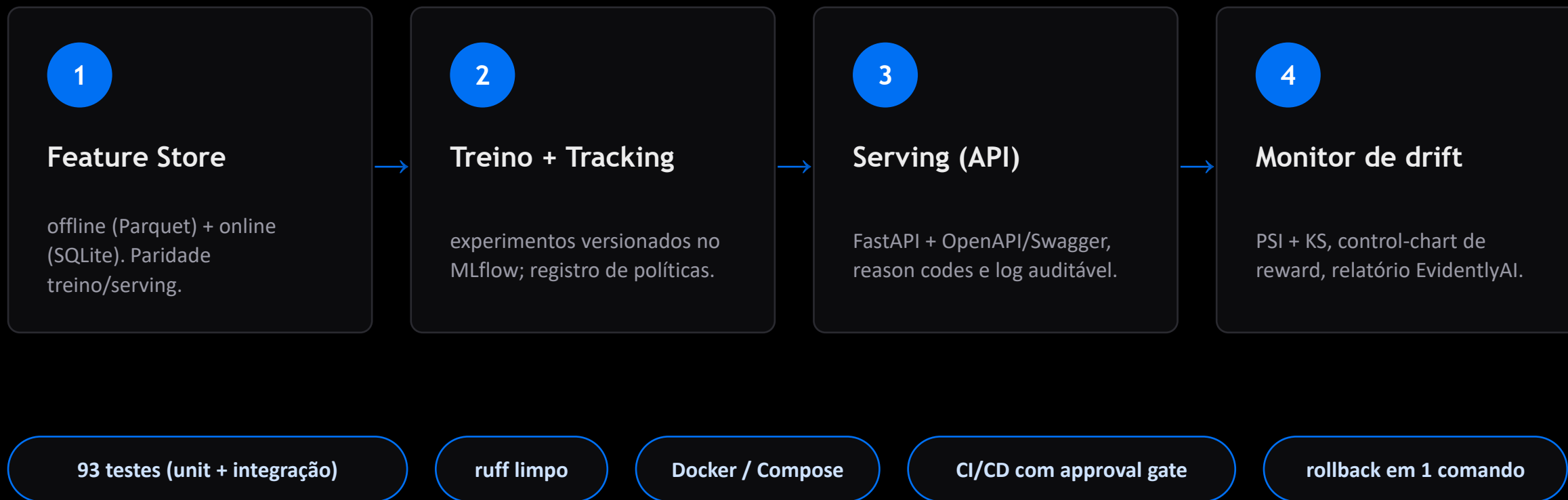
SEGURANÇA

Key Vault · Managed ID

sem segredos em env

Alternativas descartadas: AKS (overhead operacional) · segredos em env (risco) · LLM externo (viola “Azure-only”).

Da feature store ao monitoramento de drift



Responsável por design — e com olho no custo

GOVERNANÇA & RISCOS

- Model Card + System Card documentados
- Plano LGPD — sem dado pessoal real
- Humano no loop nas decisões sensíveis
- Rollback de política em 1 comando
- Riscos mapeados: reward hacking, manipulação de contexto, abuso do assistente — mitigados

FINOPS (ROI • TCO • ESCALA)

- ROI: o ganho de valor (+8–9% na base real) dilui o custo de compute/LLM
- LLM só para explicação — não está no caminho crítico da decisão
- TCO dominado por Container Apps + Azure OpenAI
- Escala ↑: réplicas + tier Redis no pico
- Escala ↓: escala a zero + cache de RAG

Como treinamos na base real

MATEMÁTICA

$$\text{value}_a = P(\text{conv} | x, a) \times \text{margin}_a$$

$$\text{regret}_T = \sum_{t=1}^T (r_t^* - r_t)$$

REFERÊNCIA

Off-policy replay sobre os eventos reais do Bank Marketing.

value = margem × conversão · regret = soma das diferenças vs. oráculo · a recompensa pode chegar atrasada (delayed reward).

```
for round_idx, i in enumerate(order):      # replay real bank-marketing events
    x, elig = contexts[i], eligibility[i]
    oracle = max(elig, key=lambda a: expected_reward(by_id[a], x)) # for regret
    decision = policy.select(x, elig)       # the bandit chooses an offer
    p = latent_conversion_prob(by_id[decision.arm_id], x)
    conv = int(rng.random() < p)           # sample the realized outcome
    reward = by_id[decision.arm_id].margin * conv # value = margin x conversion
    if conv and rng.random() < delayed_fraction:
        pending.append((round_idx + delay, arm_id, conv, x)) # delayed reward
    else:
        policy.update(arm_id, float(conv), x) # online learning
```

[src/adaptive_offers/simulation/environment.py](#)

Baseline — guloso (controle)

MATEMÁTICA

$$\hat{\mu}_a = \frac{\sum r}{n_a}$$

$$a^* = \operatorname{argmax}_a \hat{\mu}_a \times \operatorname{margin}_a$$

REFERÊNCIA

Greedy com início otimista • sem exploração ativa.

É o controle exigido pelo edital — a referência que as políticas adaptativas precisam superar.

```
def select(self, context, eligible):
    elig = self._check_eligible(eligible)
    estimates = {a: self._mean(a) for a in elig}           # running mean
    scores = {a: estimates[a] * self.margin(a) for a in elig}
    best = max(scores, key=scores.get)                     # greedy, NO explore
    return Decision(arm_id=best, explored=False, ...)
```

```
def update(self, arm_id, reward, context=None):
    self._sum[arm_id] += float(reward)
    self._count[arm_id] += 1.0
```

[src/adaptive_offers/bandits/baseline.py](#)

Thompson Sampling (Beta-Bernoulli)

MATEMÁTICA

$$\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$$

$$a^* = \operatorname{argmax}_a \theta_a \times \text{margin}_a$$

$$\alpha_a \leftarrow \alpha_a + r, \quad \beta_a \leftarrow \beta_a + 1 - r$$

REFERÊNCIA

Thompson (1933) • Agrawal & Goyal (2012).

Amostrar do posterior É a exploração: braços com pouca evidência têm posterior largo e às vezes são sorteados alto.

```
def select(self, context, eligible):
    elig = self._check_eligible(eligible)
    samples = {a: self.rng.beta(self.alpha[a], self.beta[a]) for a in elig}
    scores = {a: samples[a] * self.margin(a) for a in elig} # margin-weighted
    best = max(scores, key=scores.get) # Thompson draw
    return Decision(arm_id=best, explored=..., ...)
```

```
def update(self, arm_id, reward, context=None):
    r = float(np.clip(reward, 0.0, 1.0))
    self.alpha[arm_id] += r # Beta-Bernoulli conjugacy
    self.beta[arm_id] += 1.0 - r
```

[src/adaptive_offers/bandits/thompson.py](#)

Nilos-UCB — UCB-V (consciente de variância)

MATEMÁTICA

$$\text{idx}_a = \mu_a + \sqrt{\frac{2 v_a \ln t}{n_a}} + c \frac{3 \ln t}{n_a}$$

$$a^* = \operatorname{argmax}_a \text{idx}_a \times \text{margin}_a$$

REFERÊNCIA

Audibert, Munos & Szepesvári (2009) — UCB-V.

O bônus usa a variância empírica do braço (Welford online); ofertas de alta incerteza são exploradas mais cedo.

```
def _index(self, arm_id, t):
    # UCB-V (variance-aware)
    n = self.counts[arm_id]
    if n == 0:
        return math.inf
    # force one pull per arm
    ln_t = math.log(max(t, 2))
    v = self.variance(arm_id)
    return self.mean[arm_id] + math.sqrt(2*v*ln_t/n) + self.c*3*ln_t/n

def update(self, arm_id, reward, context=None):
    # Welford online variance
    self.counts[arm_id] += 1; n = self.counts[arm_id]
    delta = reward - self.mean[arm_id]
    self.mean[arm_id] += delta / n
    self._m2[arm_id] += delta * (reward - self.mean[arm_id])
```

[src/adaptive_offers/bandits/ucb.py](#)

LinUCB — bandit contextual (menor regret)

MATEMÁTICA

$$A_a = I_d + \sum x x^T, \quad b_a = \sum r x$$

$$\theta_a = A_a^{-1} b_a, \quad p_a = \theta_a^T x$$

$$\text{ucb}_a = p_a + \alpha \sqrt{x^T A_a^{-1} x}$$

$$a^* = \operatorname{argmax}_a \text{ucb}_a \times \text{margin}_a$$

REFERÊNCIA

Li, Chu, Langford & Schapire (2010).

Ridge por braço sobre o contexto + bônus de incerteza → roteia a oferta certa para o perfil certo. Na base real teve o menor regret (8,3%) e a maior conversão (9,1%); ganho de valor modesto (~+8%).

```
def _predict(self, arm_id, x):
    # ridge regression per arm
    A_inv = np.linalg.inv(self.A[arm_id])
    theta = A_inv @ self.b[arm_id]
    mean = float(theta @ x) # predicted P(conversion)
    bonus = self.alpha * np.sqrt(x @ A_inv @ x) # uncertainty bonus
    return mean, bonus
```

```
def update(self, arm_id, reward, context):
    x = np.asarray(context, float)
    self.A[arm_id] += np.outer(x, x) # A += x x^T
    self.b[arm_id] += float(reward) * x # b += r x
```

[src/adaptive_offers/bandits/linucb.py](#)

MLflow — Rastreamento de Experimentos

Reprodutibilidade · Comparação de Políticas · Registro de Modelos

OBSERVABILIDADE E GOVERNANÇA DE ML

Experiment Tracking

1 run = 1 politica simulada

params: horizon, alpha, beta

metricas: reward, regret

tags: policy, stage, seed

Model Registry

LinUCB e a politica ativa (v1)

registry em artifacts/policies

rollback em 1 comando

API sempre le a versao ativa

Resultados-chave

LinUCB: regret ratio 8,3%

Lift +8,2% vs baseline

DR-OPE 19,18 [17,66-20,65]

6.000 rounds, dados reais

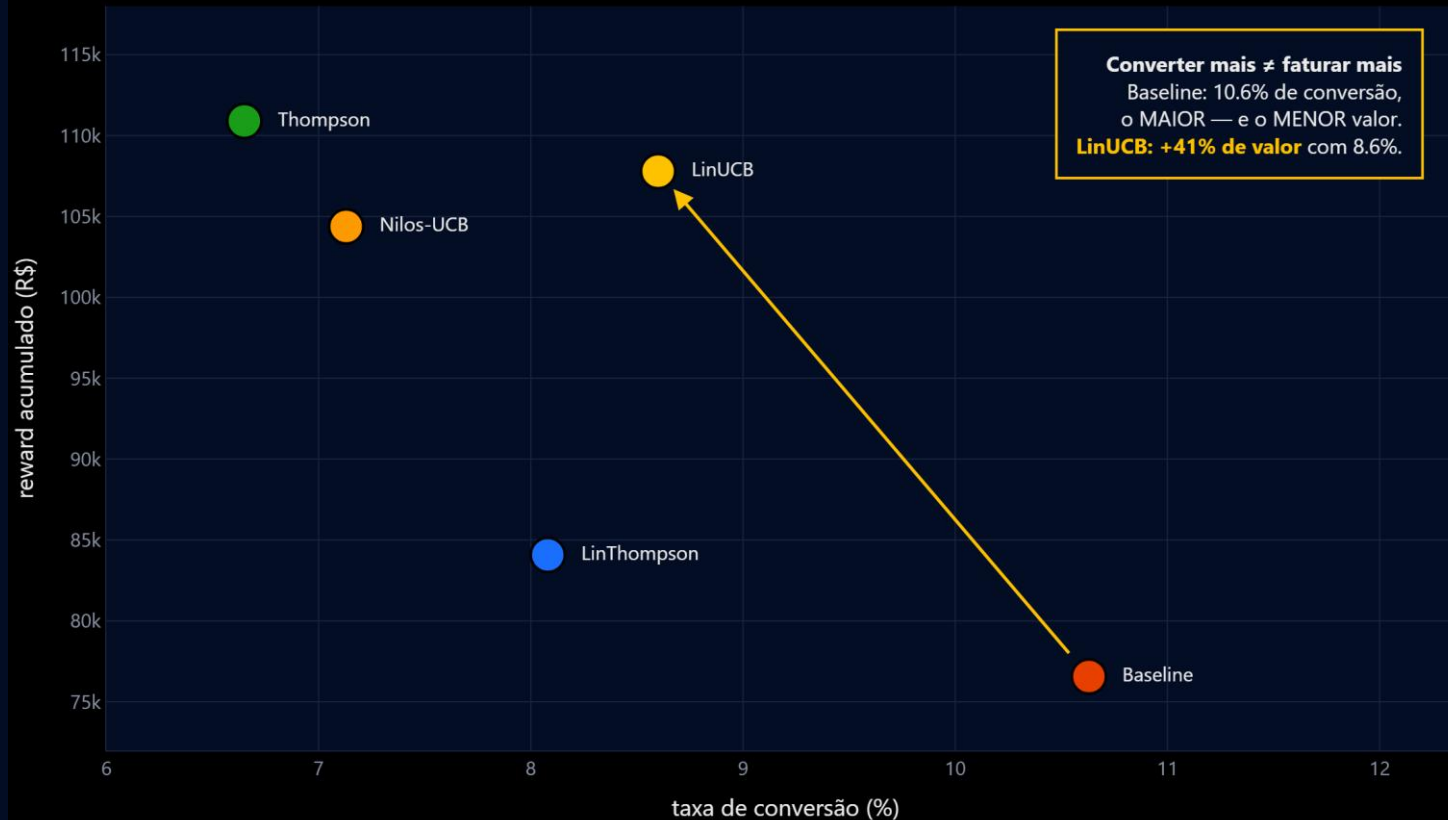
Política	Reward Acum.	Regret	Conversao	Lift vs Baseline
Thompson	R\$ 114.290	11,8%	7,1%	+9,2%
LinUCB (recomendada)	R\$ 113.230	8,3%	9,1%	+8,2%
Baseline (greedy)	R\$ 104.700	10,9%	6,2%	---
Nilos-UCB	R\$ 102.020	17,0%	7,1%	-2,6%

MLflow — Conversão não é lucro

O experimento que sustenta a tese do pitch — leitura direta dos runs

Valor acumulado × taxa de conversão

MLflow · 6.000 rounds · seed=42 · Bank Marketing (UCI)



A LEITURA

O Baseline tem a **MAIOR conversão (10,6%)** e o **MENOR valor (R\$ 76.560)**.

Ele empurra quase tudo para a oferta de maior margem e converte em volume — mas ignora o contexto e queima as decisões de maior valor esperado.

O LinUCB converte **MENOS (8,6%)** e entrega +41% de valor: escolhe a oferta certa para o perfil certo, não a mais fácil de vender.

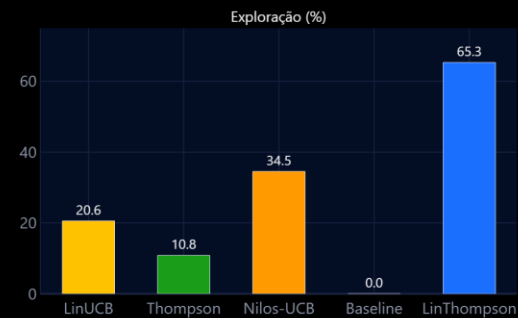
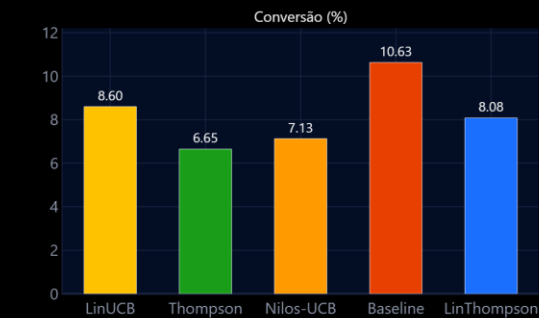
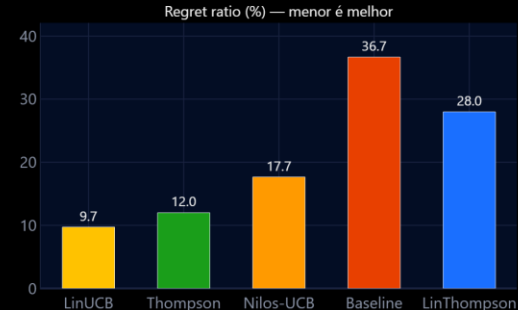
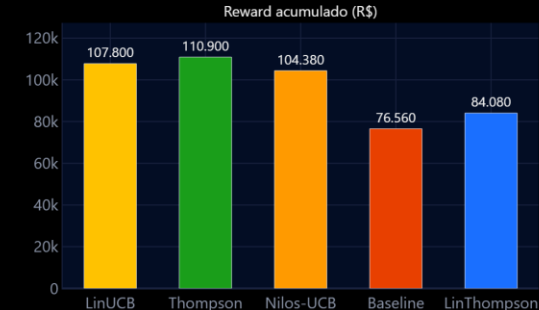
Por isso a recompensa é ponderada por margem:
 $\text{value} = P(\text{conversão}) \times \text{margem}.$

MLflow — Comparação e robustez

As quatro métricas que a banca olha — e por que 1 seed não decide nada

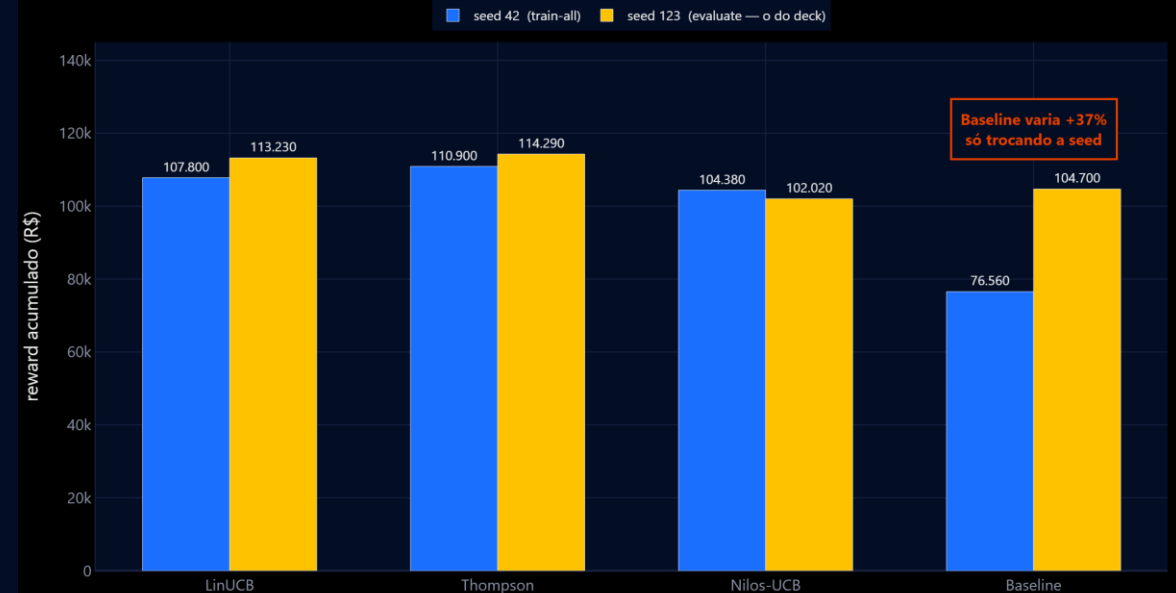
Comparação de políticas — runs do MLflow

6.000 rounds · seed=42 · 1 run por política



A mesma política, duas seeds

6.000 rounds · por isso a conclusão do relatório usa 5 seeds, não uma



O QUE ISSO SIGNIFICA

LinUCB tem o MENOR regret (9,7%) — e o baseline, sem exploração, o maior (36,7%). Explorar ~20% do tráfego se paga.

Trocando só a seed, o baseline varia +37%. Por isso a conclusão do relatório usa 5 seeds: LinUCB lidera na média, com CV de 2,97%.

API REST — Servico de Decisao

FastAPI · 14 endpoints · auditavel · localhost:8000/docs

CONTRATO DE DECISAO EM TEMPO REAL

	/health	Liveness + readiness
GET	/policy	Politica ativa - nome, versao, metricas
GET	/offers	Catalogo de 6 ofertas - margem, suitability
POST	/decide	Contexto -> decisao auditavel + reason_codes
POST	/assistant/explain	Decide + LLM+RAG - resposta em linguagem natural
GET	/metrics	Matriz de comparacao de politicas
GET	/metrics/regret-curve	Curvas de regret acumulado para visualizacao

FLUXO DE DECISAO

POST /decide -> Feature Extraction -> Eligibility Guard -> Bandit Policy -> Reward Estimate -> Audit Log -> DecisionOut

DASHBOARD BI — Observability Board

Streamlit · Tempo real · Multi-armed bandit · localhost:8503

O QUE O DASHBOARD FAZ

⚡ Métricas em Tempo Real

- 4 KPI tiles com sparklines de aprendizado
- Reward/1k, Regret ratio, Conversão, Lift
- Atualiza a cada 30s via @st.fragment
- Mostra política ativa e rodadas simuladas

🦉 Feed de Decisões ao Vivo

- Log de auditoria em tempo real (audit.jsonl)
- Coloração: Verde = Exploit · Ciano = Explore
- Timestamps em horário local (UTC-3)
- 112+ decisões registradas e auditáveis

🤖 Assistente LLM + RAG

- Explica cada decisão em linguagem natural
- RAG recupera política comercial por semântica
- Modo offline (análise ML) sem custo de API
- Modo online com Claude via Anthropic API

MAIS FUNCIONALIDADES

📊 Dinâmica de Aprendizado

Heatmap multi-métrica, curvas de regret por janela, lollipop de exploração, barras de pulls por oferta

🔧 Explorador de Decisão

Simula uma decisão ao vivo via API com contexto do cliente, exibe oferta elegível com análise de valor esperado

📈 Rastreo de Execução

Trace de RAG (ms), LLM (ms), total (ms), chunks recuperados, query semântica e snippet do prompt

O IMPACTO

Decide, aprende e se explica.

Escolhe em tempo real a oferta de maior valor esperado para cada cliente — com governança pronta para produção e decisões auditáveis.

+9%

melhor vs. baseline

8,3%

menor regret

100%

decisões auditáveis